

面向预测不确定性的微网购电与储能滚动调度

摘要

本文研究含光伏、储能与外部电网的微网购电调度问题。首先统一建立以 10 分钟为步长的能量平衡与储能状态线性规划模型，并通过数据门禁核验 365 日、144 时段的负荷、光伏和电价数据。问题一在确定性曲线下求解日内调度，购电费为 35,126.95 元，较无储能基线 48,052.05 元降低 26.90%。问题二采用严格向前滚动的同星期中位数与最近 7 日中位数融合预测，并叠加历史净负荷残差 80% 分位安全裕度；334 日回测费用为 15,391,996.17 元，较简单点预测无储能基线降低 22.96%，紧急购电由 739,783.33 kWh 降至 427,994.14 kWh。问题三在 0:00、6:00、12:00、18:00 更新剩余时段，冻结已执行决策并继承 SOC；滚动策略将紧急购电降至 410,727.00 kWh，但因调整费使总费用升至 16,258,910.51 元，高于不调整方案 15,818,394.81 元。问题四以历史价格滚动预测扩展模型，价格预测 MAE 为 0.0504 元/kWh；问题 4-2 费用为 16,163,346.40 元，问题 4-3 费用为 16,977,176.53 元。全部 1,337 项约束回代均通过。结果表明，储能削峰填谷与分位裕度能有效降低费用和缺电风险，但更频繁的信息更新只有在预测收益超过调整罚金时才具有经济价值。

关键词：微网调度 线性规划 滚动预测 储能 鲁棒裕度

一、 问题重述

某微网由小区负荷、光伏发电、储能装置及外部电网构成。外部电网负责补足本地供能缺口，储能可在低价或光伏富余时充电，在高价或供电不足时放电。题目给出固定日内电价与典型日曲线、全年负荷和光伏实测数据、每日四次光伏预报以及全年波动电价，要求制定经济且可执行的购电策略。

问题一要求在负荷、光伏和电价均已知时安排 144 个 10 分钟时段的购电与储能动作；问题二要求在每日 0:00 仅凭历史信息制定全天计划，并以 5 倍电价补充紧急购电；问题三允许在 6:00、12:00 和 18:00 依据新增预报调整尚未执行的购电量；问题四用波动电价分别重算问题二和问题三。各问共享储能物理约束，但信息集和结算方式逐层扩展。

本文总体技术路线见图 1。核心原则是先统一数据口径和物理可行域，再逐层增加预测误差、滚动更新与价格波动，最后以独立回代和简单基线检验模型收益。

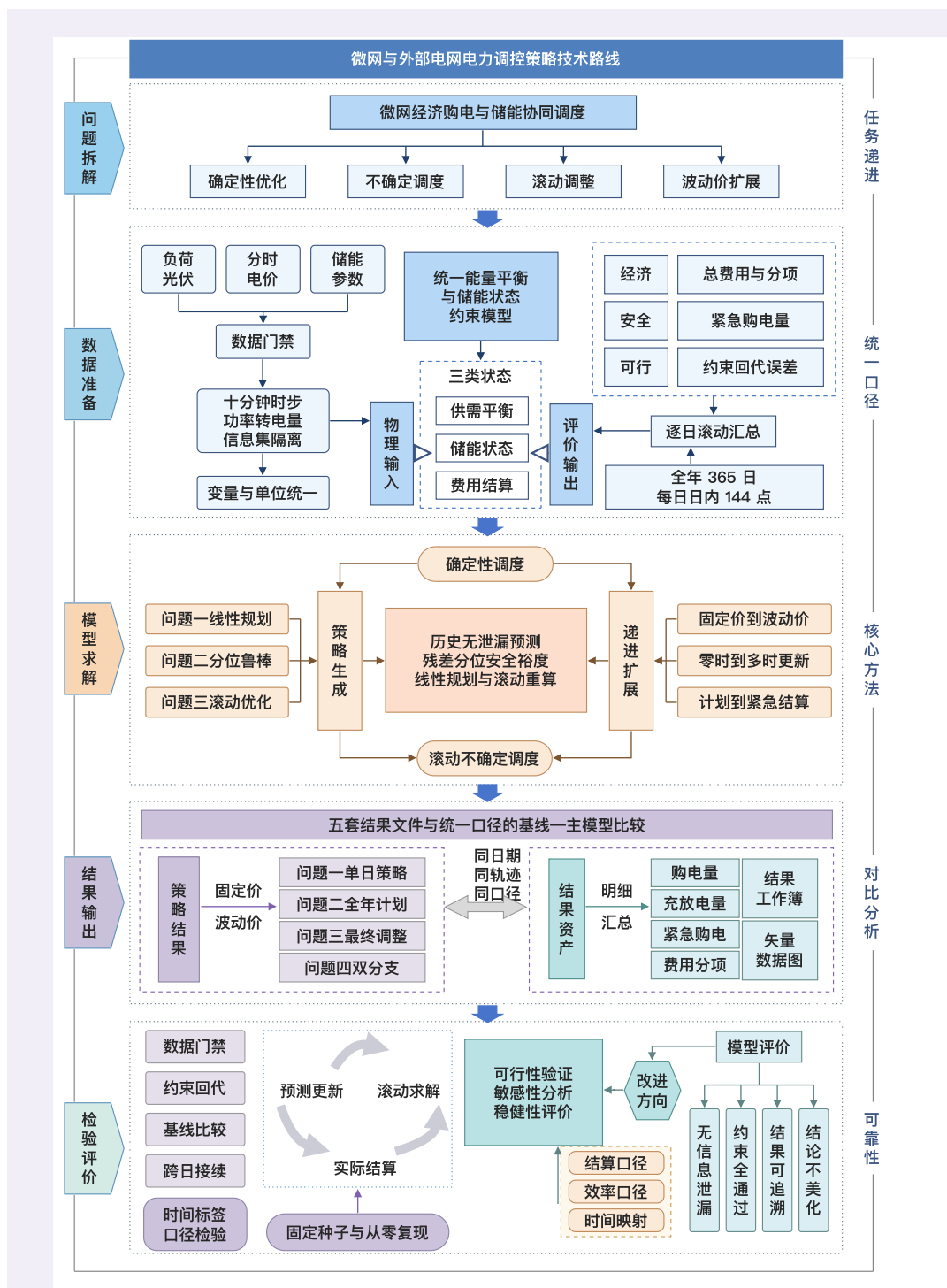


图 1: 微网购电与储能协同调度总体技术路线

二、 数据处理与问题分析

2.1 数据门禁与统一口径

全年负荷、光伏和波动电价数据均为 365×144 ，每日四次光伏预报形成 1,460 个唯一“日期-发布时刻”键。检查结果为数值缺失 0、负值 0、重复日期 0。负荷范围为

1,995.7176–7,978.8849 kW, 光伏为 0–10,216.2000 kW, 波动电价为 0.0076–1.7936 元/kWh。夜间光伏零值具有物理意义, 予以保留; 光伏预报表的日期结构空白采用向下填充恢复。功率进入能量平衡前按

$$\Delta t = \frac{1}{6} \text{ h}, \quad E_{d,t} = P_{d,t} \Delta t \quad (1)$$

转换为时段电量。原始 144 点按官方结果模板的行序一一对应, 另以整体平移 ± 10 分钟检验标签歧义。决策日 d 的模型只能读取 d 以前的实测数据及决策时已经发布的预报, 事后实测轨迹仅用于结算, 避免未来信息泄漏。数据流程见图 2。

数据门禁与无泄漏建模流程

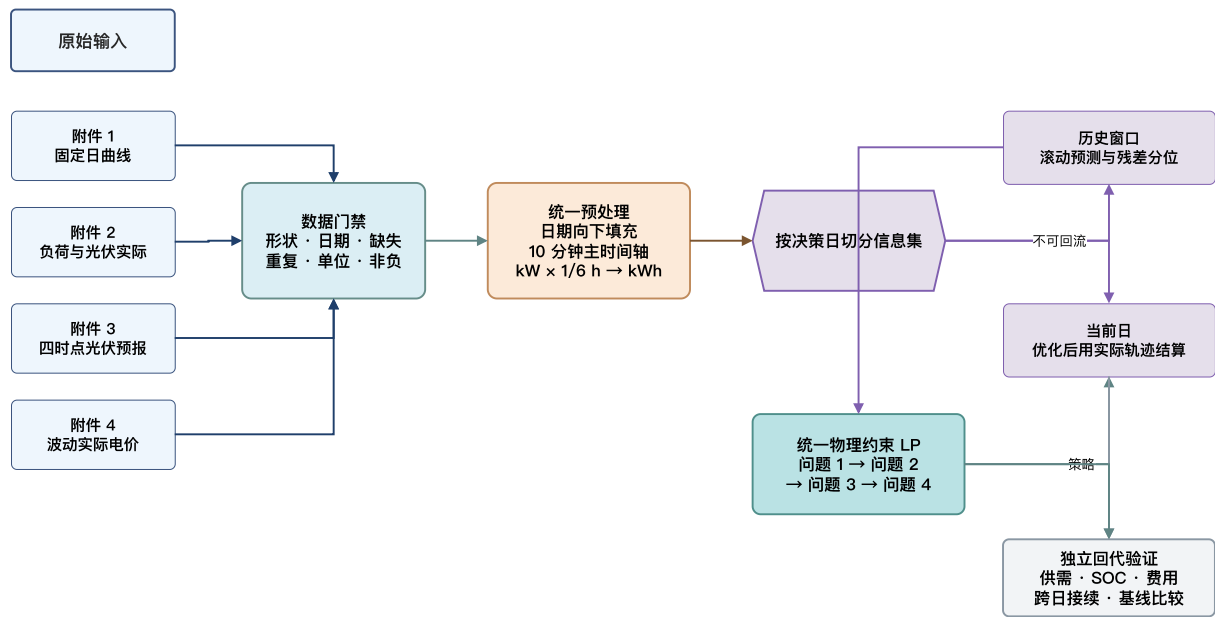


图 2: 数据门禁、信息集切分与无泄漏建模流程

2.2 四问递进关系

问题一提供确定性线性规划基座; 问题二加入预测与紧急购电; 问题三只对尚未执行时段滚动重算, 并把“不调整”保留为可行基线; 问题四只改变电价过程和其可见信息, 不改变能量平衡。这样可将性能差异归因于新增信息或价格机制, 而不是模型口径变化。

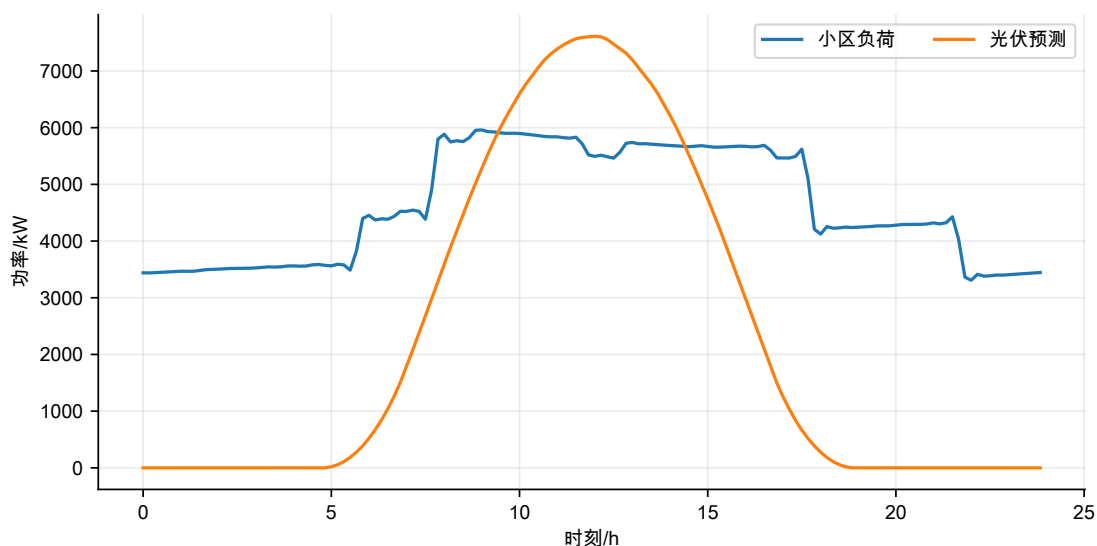


图 3: 附件一负荷与光伏功率的日内变化

三、 模型假设

1. 每个 10 分钟时段内负荷、光伏功率和电价保持常数，功率按式 (1) 转换为电量。
2. 储能充电效率和放电效率均为 0.9，容量安全区间为 $[1200, 10800]$ kWh，交流侧充、放电功率上限均为 5,000 kW。
3. 微网不向外网售电；供给过剩允许弃置并显式记录。题目未给出并网购电功率上限，故不额外添加。
4. 问题一固定期初、期末 SOC 均为 6,000 kWh；问题二至问题四跨日继承 SOC，并要求每日优化终点不低于 6,000 kWh。
5. 决策时刻之后的实际负荷、光伏和波动电价不可见。所有预测由历史日顺序生成，不进行随机训练集划分。
6. 调整费用按“原计划全额，加相对 0:00 原计划的一次偏差费，再加紧急购电费”计算；已经执行的时段不可修改。

四、 符号说明

表 1: 主要符号与单位

符号	含义	单位
d, t, r	日期、10 分钟时段、发布时刻	日, -, h
$L_{d,t}, G_{d,t}$	负荷、光伏功率	kW
$\pi_{d,t}$	电价	元/kWh
$p_{d,t}, g_{d,t}, e_{d,t}$	原计划、最终正常购电、紧急购电	kWh
$c_{d,t}, q_{d,t}$	充电量、放电量	kWh
$S_{d,t}$	时段末储能电量	kWh
$w_{d,t}$	弃光或过剩电量	kWh
η_c, η_d	充、放电效率, 均取 0.9	-
$\rho_{d,t}$	历史净负荷残差 80% 分位裕度	kWh

五、 问题一：确定性购电与储能调度

5.1 统一物理模型

令净负荷电量为 $n_t = (L_t - G_t)\Delta t$ 。供需与 SOC 递推写为

$$g_t + q_t - c_t \geq n_t, \quad (2)$$

$$S_t = S_{t-1} + \eta_c c_t - q_t / \eta_d. \quad (3)$$

约束为 $1200 \leq S_t \leq 10800$, $0 \leq c_t, q_t \leq 5000/6$, 且 $S_0 = S_{144} = 6000$ 。代码以不等式 (2) 表示可弃置的富余供给, 等价于加入非负弃电变量后写等式。目标函数为

$$\min \sum_{t=1}^{144} \pi_t g_t + 10^{-7} \sum_{t=1}^{144} (c_t + q_t), \quad (4)$$

其中极小正则项只用于排除退化的同时充放电, 不改变两位小数费用结论。模型由 HiGHS 线性规划求解; 其底层高性能对偶修正单纯形方法见文献 [3]。求解后独立回代供需、SOC 和充放电互斥。

5.2 结果与基线比较

主模型购电费为 35,126.95 元, 全天购电 59,482.70 kWh; 无储能直接购电基线为 48,052.05 元, 节省 26.90%。储能充电 20,740.67 kWh、放电 16,799.94 kWh, SOC 由 6,000 kWh 回到 6,000 kWh, 且始终位于 1,200–10,800 kWh。图 4 表明, 储能在低价或光伏富余时段吸收电量, 并在高价时段释放。

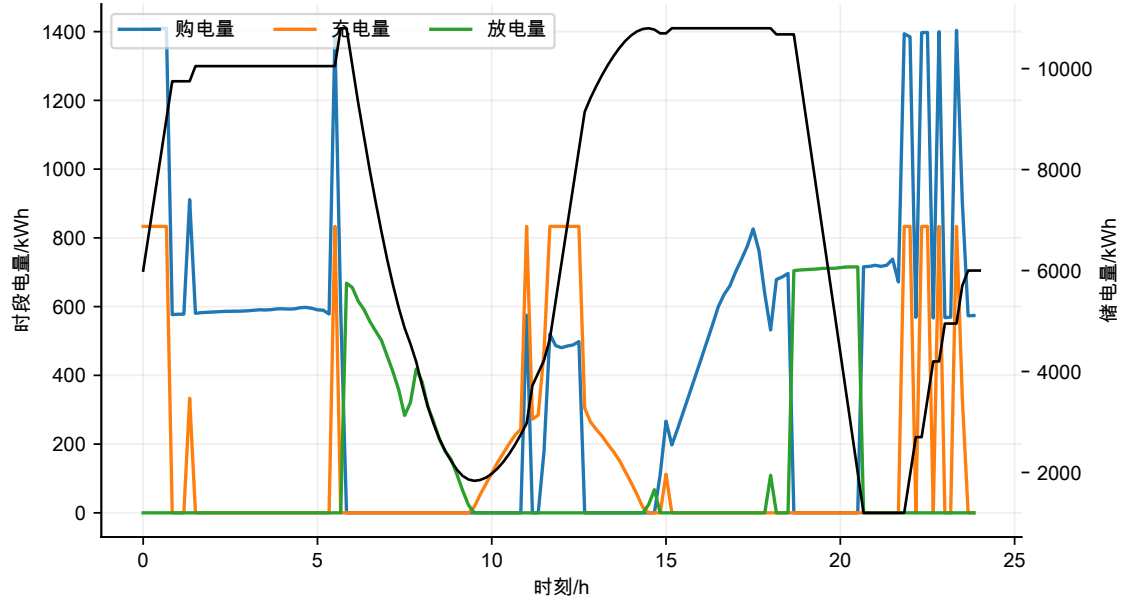


图 4: 问题一购电、充放电与 SOC 轨迹

六、 问题二：无泄漏预测与分位鲁棒调度

6.1 滚动预测

从 2025 年 2 月 1 日起逐日回测，1 月仅作为历史初始化。对变量 $x \in \{L, G, \pi\}$ ，同一时段的点预测为

$$\hat{x}_{d,t} = 0.65 \text{ med}\{x_{j,t} : j < d, j \equiv d \pmod{7}\} + 0.35 \text{ med}\{x_{j,t} : d - 7 \leq j < d\}. \quad (5)$$

同星期部分最多取最近 4 个同星期样本。负荷预测 MAE 和 WAPE 分别为 303.10 kW 和 6.57%；光伏分别为 208.60 kW 和 8.77%。图 5 展示一日预测实例。

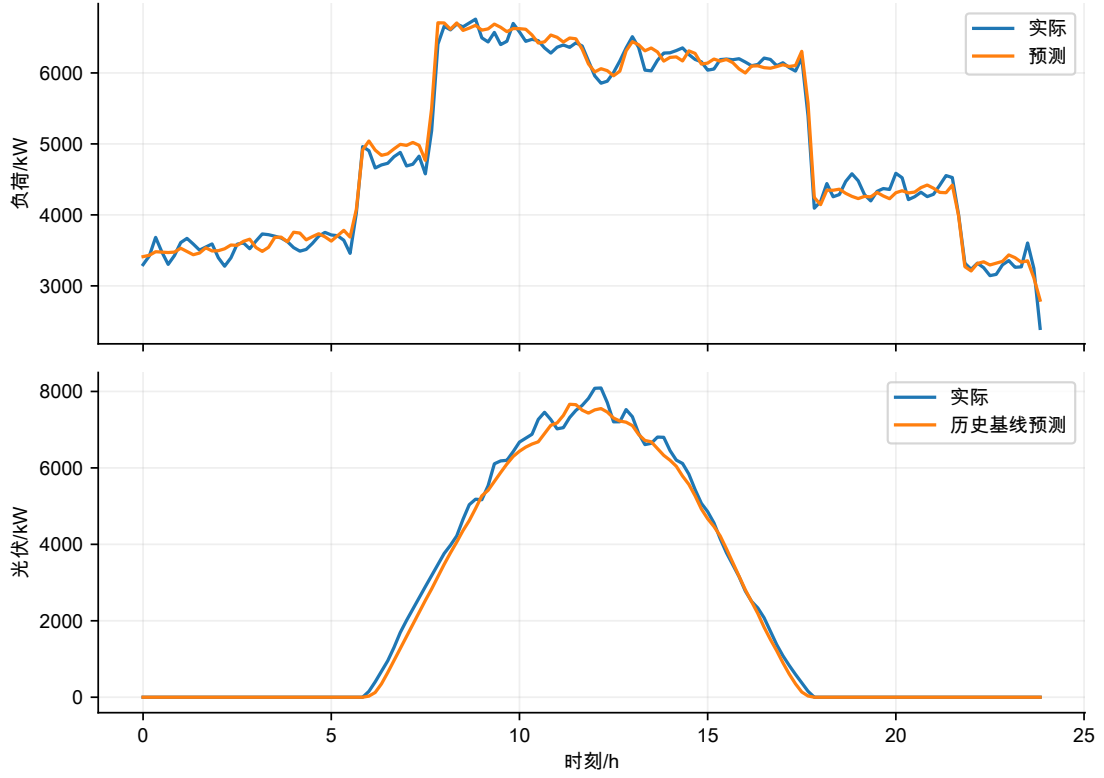


图 5: 负荷与光伏历史滚动预测示例

6.2 安全裕度与优化

将历史 28 日内有效预测残差 $u_{j,t} = n_{j,t} - \hat{n}_{j,t}$ 的 80% 分位记为 $\rho_{d,t}$, 以

$$\tilde{n}_{d,t} = \hat{n}_{d,t} + \rho_{d,t} \quad (6)$$

代入问题一的线性规划。实际供给不足部分

$$e_{d,t} = \max\{n_{d,t} - (g_{d,t} + q_{d,t} - c_{d,t}), 0\} \quad (7)$$

按 $5\pi_{d,t}$ 结算。该模型是场景风险模型的线性分位近似, 代码并未计算 CVaR 尾部值, 故本文不将结果表述为完整 CVaR 优化。

6.3 结果

334 日累计费用为 15,391,996.17 元, 简单点预测且不使用储能的基线为 19,978,861.43 元, 下降 22.96%。紧急购电由 739,783.33 kWh 降至 427,994.14 kWh, 说明分位裕度与储能协同同时改善经济性和缺电风险。

七、 问题三：多时点滚动调整

7.1 滚动机制与费用

0:00 产生原计划 $p_{d,t}$, 6:00、12:00、18:00 将小时光伏预报线性插值到 10 分钟, 并用当日已观测负荷偏差修正剩余时段负荷。每次只优化未执行区间, 冻结历史决策并继承上一时段 SOC。这一做法属于微网能量管理中常见的滚动时域控制框架^[1-2], 具体机制如图 6。

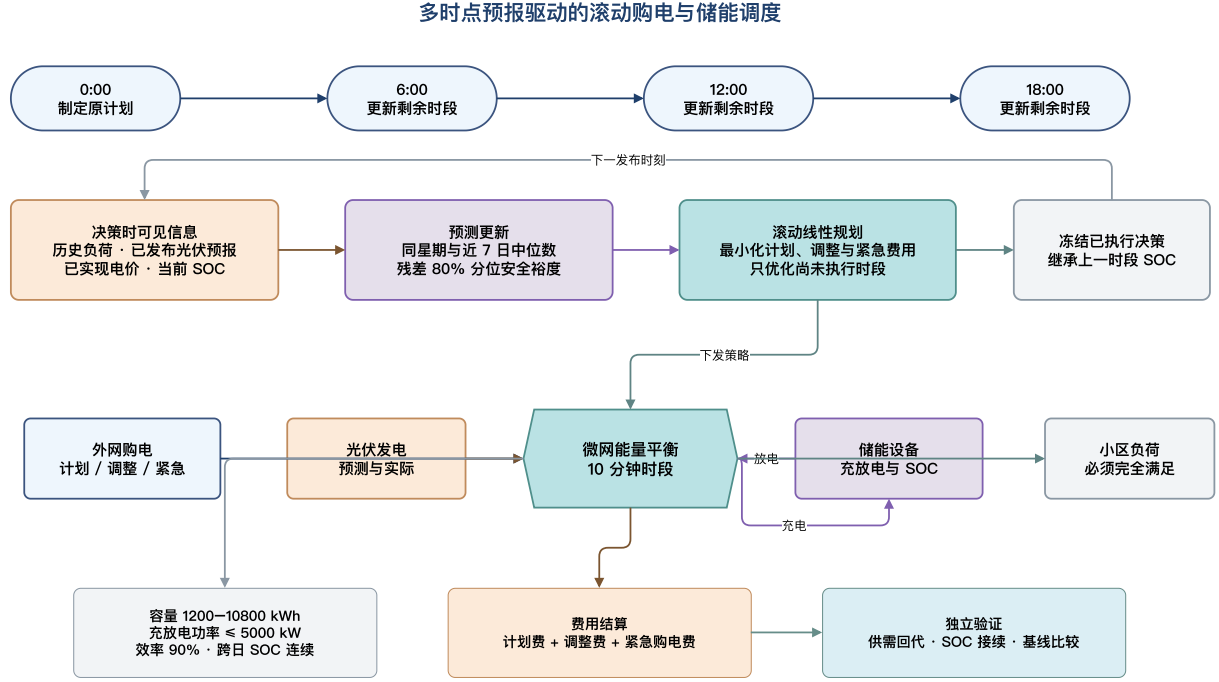


图 6: 多时点光伏预报驱动的滚动购电与储能调度机制

最终正常购电量为 $g_{d,t}$ 。按采用的结算解释, 每日总费用为

$$C_d = \sum_t [\pi_t p_t + 1.5\pi_t (g_t - p_t)^+ + 0.5\pi_t (p_t - g_t)^+ + 5\pi_t e_t]. \quad (8)$$

调整阶段的 LP 以相对原计划的上调量、下调量作为辅助变量, 并保留问题一的 SOC 和功率约束。

7.2 真实效果及其解释

全滚动策略费用为 16,258,910.51 元, 不调整基线为 15,818,394.81 元, 即费用增加 2.78%。与此同时, 紧急购电由 477,723.73 kWh 降至 410,727.00 kWh。紧急购电虽下降, 但其费用收益不足以抵消调整代价。因此“预报更新降低物理缺口”并不等于“提高经济收益”。逐日差异分布见图 7。

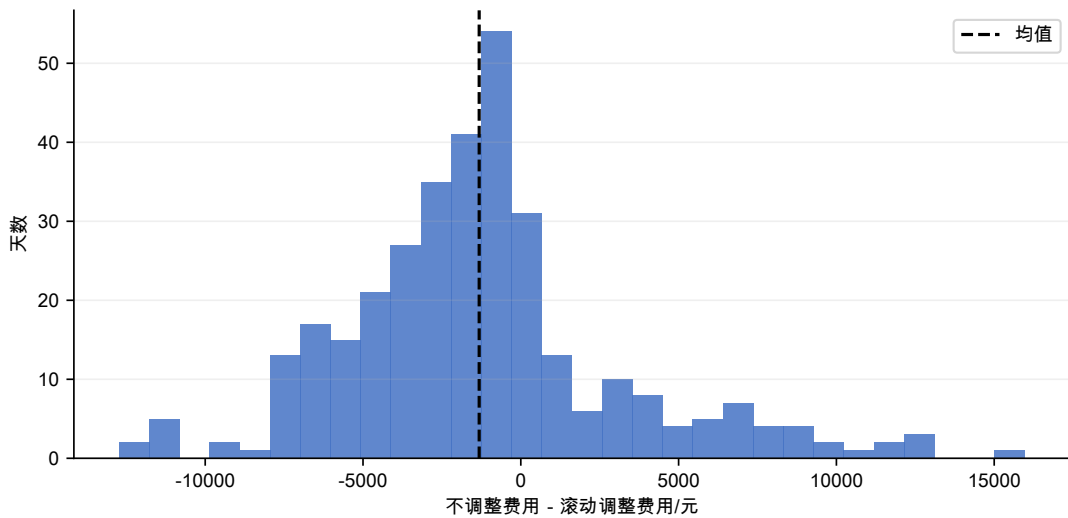


图 7: 问题三不调整费用与滚动调整费用的逐日差值分布

八、 问题四：波动电价下的扩展

8.1 价格预测与问题 4-2

未来实际价格不可见，故对电价同样采用式 (5) 的历史滚动预测，实际价格只用于事后结算。价格预测 MAE 和 WAPE 分别为 0.0504 元/kWh 和 6.66%，预测与实际日均价格见图 8。

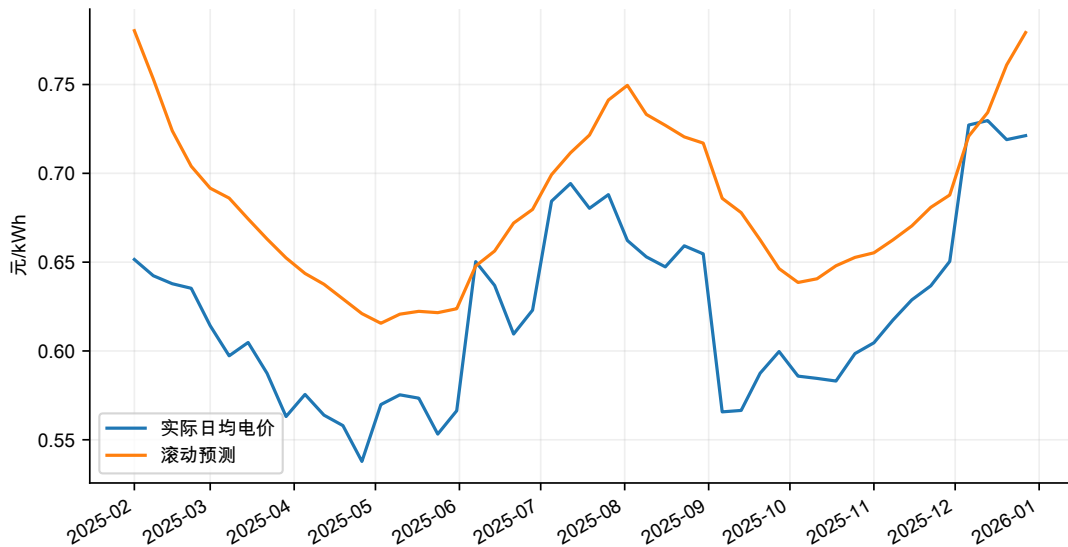


图 8: 波动电价的历史滚动预测与实际值

问题 4-2 将预测价格代入问题二的目标函数，实际价格结算后费用为 16,163,346.40 元；将问题二固定价策略直接用于波动价的基线为 16,171,689.06 元，前者降低 8,342.66 元，约 0.05%。改进幅度较小，表明当前简单价格预测对调度顺序的边际影响有限。

8.2 问题 4-3 与综合比较

问题 4-3 同时在发布时点更新光伏预测，并根据已观察价格偏差修正剩余时段的价格预测。滚动策略费用为 16,977,176.53 元，不调整基线为 16,567,282.25 元，滚动调整增加 409,894.28 元，说明当前结算规则下更新带来的收益仍不足以覆盖调整代价。图 9 汇总四组主模型与基线。问题二和问题 4-2 改善费用，而问题三和问题 4-3 说明高调整罚金下全时点更新并非必然最优。

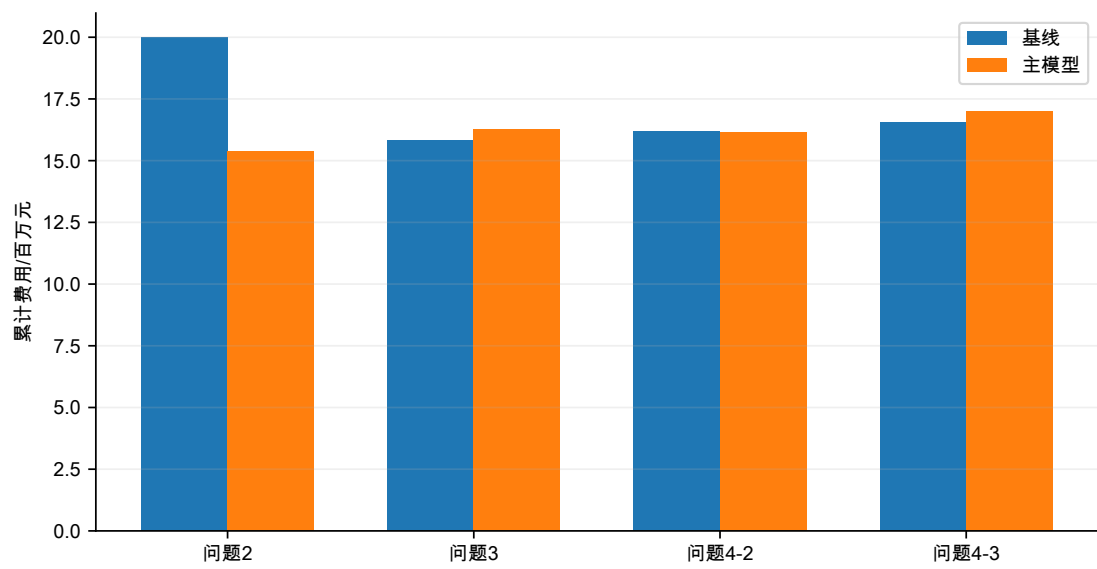


图 9: 各问题主模型与对应基线累计费用比较

表 2: 四个指定日期的主模型总费用（元）

日期	问题二	问题三	问题 4-2	问题 4-3
2025-03-20	45,655.77	57,498.21	46,925.61	59,045.90
2025-06-21	31,280.83	31,320.96	26,760.27	26,986.85
2025-09-23	56,667.56	51,011.60	59,231.53	53,384.07
2025-12-21	66,260.98	69,667.17	78,128.57	82,315.16

九、 敏感性、稳健性与一致性检验

9.1 时间标签敏感性

将问题一的 144 点净负荷与电价整体平移 -10、0、+10 分钟并重新优化，费用分别为 35,129.73、35,126.95 和 35,126.85 元；相对主口径最大偏差仅 2.78 元，说明总费用对这一行序歧义不敏感。指定时段的解释仍应遵循官方模板行序。

9.2 可行性与数值残差

求解前以零储能、直接购电构造可行基线；求解后独立回代供需、SOC、容量边界及费用分量。问题一与 334 日滚动任务共 1,337 项检查全部通过。最小供给松弛为 -1.137×10^{-13} kWh，最大 SOC 递推残差为 1.819×10^{-12} kWh，最大同时充放电量为 0；所有 SOC 均位于 [1200, 10800] kWh。上述误差处于浮点求解容差内。

9.3 结果稳健性边界

问题二的结论同时体现在费用下降 22.96% 和紧急购电下降 42.15%，方向一致。问题三及问题 4-3 则表现为紧急购电下降、费用上升，说明其结论依赖调整费结构。当前实现仅比较“全部更新”与“完全不调整”两个端点，尚未穷举 0/6、0/12、0/18 等发布组合；完整场景 CVaR、不同调整结算解释和充放电效率口径应作为后续扩展，而不能据现有结果虚构敏感性数值。

十、模型评价与结论

10.1 模型优点

本文以同一能量平衡和 SOC 递推贯穿四问，新增机制只改变信息集和费用项，便于归因比较。预测按日期严格前推，避免随机划分造成时间泄漏；优化前验证可行性，优化后独立回代 1,337 次并保存迭代记录。分位裕度将紧急购电代价转化为可解释的保守净负荷，计算规模适合全年多次滚动求解。

10.2 模型局限

分位鲁棒 LP 是完整场景风险模型的近似，未显式描述时序相关场景或输出 CVaR。储能互斥依赖正电价、效率损耗和极小吞吐正则项，而非二进制变量；虽然实际检查未出现同时充放电，但在允许负电价或售电时需改为混合整数模型。滚动调整仅比较两个端点策略，尚不能确定最优发布频率。

10.3 结论与推广

确定性条件下，储能低充高放可使典型日费用下降 26.90%；不确定条件下，历史滚动预测与 80% 残差分位裕度使 334 日费用下降 22.96%。多时点更新能减少紧急购电，却可能因调整罚金提高总费用；波动电价预测在问题 4-2 中仅带来约 0.05% 的费用改善。因此实际运营应把“是否调整”作为带成本的决策，而不是收到新预报后机械重算。该框架可推广至园区综合能源、充电站和含可再生能源的企业微网，只需替换设备约束、结算规则和预测模块。

十一、 人工智能工具使用声明

本队在竞赛期间使用了人工智能工具，主要用于文字润色、代码调试与排版检查。详细使用情况见支撑材料中的《人工智能工具使用详情》。

十二、 参考文献

- [1] Prodan I, Zio E. A model predictive control framework for reliable microgrid energy management[J]. International Journal of Electrical Power & Energy Systems, 2014, 61: 399–409. DOI: 10.1016/j.ijepes.2014.03.017.
- [2] Hu J, Shan Y, Guerrero J M, et al. Model predictive control of microgrids—An overview[J]. Renewable and Sustainable Energy Reviews, 2021, 136: 110422. DOI: 10.1016/j.rser.2020.110422.
- [3] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142. DOI: 10.1007/s12532-017-0130-5.

附录：核心实现与复现

支撑材料文件列表

支撑材料压缩包包含以下内容：

- `code/run_all.py`（完整可运行源程序）与 `code/README.md`（环境和运行说明）；
- `code/outputs/records/`（参数、预测、逐日指标和约束检查）与 `code/outputs/results/`（五个结果工作簿）；
- `figures/`（论文图及其 CSV 数据源）和《人工智能工具使用详情》PDF。

赛题给出的原始附件不重复收入支撑材料。

复现说明与核心接口

本文结果由同一程序从原始工作簿生成。随机种子固定为 20260910，核心参数为 $\Delta t = 1/6$ 小时、 $\eta_c = \eta_d = 0.9$ 、 $S \in [1200, 10800]$ kWh、充放电功率上限 5,000 kW、滚动残差分位数 0.80。核心 LP 接口如下：

```
def optimize_schedule(net_kwh, price, s0,
                      terminal_eq=None, terminal_min=6000,
                      plan_ref=None):
    # Variables: grid, charge, discharge, SOC.
    # Adjustment mode adds upward/downward deviations.
    res = linprog(cobj, A_ub=Aub, b_ub=bub,
                  A_eq=Aeq, b_eq=beq, bounds=bounds,
                  method="highs",
                  options={"dual_feasibility_tolerance": 1e-8,
                           "primal_feasibility_tolerance": 1e-8})
    if not res.success:
        raise RuntimeError(res.message)
    return checked_schedule(res.x)
```

在项目根目录执行以下命令可从零读取附件、完成数据门禁、求解四问、回代约束并重建结果工作簿与数据图：

```
MPLCONFIGDIR=.mplconfig python3 code/run_all.py
```

在 `paper` 目录连续执行两次以下命令可重建论文：

```
xelatex -interaction=nonstopmode main.tex
xelatex -interaction=nonstopmode main.tex
```

完整源程序

以下为生成全部结果、记录与数据图的完整可运行源程序。

```

#!/usr/bin/env python3
"""Reproducible implementation for C题微网调度 (seed=20260910)."""
from __future__ import annotations

import argparse
import json
import platform
import shutil
import sys
import time
from copy import copy
from dataclasses import dataclass
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import openpyxl
import pandas as pd
import scipy
from openpyxl import load_workbook
from scipy.optimize import linprog

SEED = 20260910
DT = 1.0 / 6.0
T = 144
ETA_C = 0.9
ETA_D = 0.9
SOC_MIN = 1200.0
SOC_MAX = 10800.0
SOC_SAFE = 6000.0
P_MAX_KWH = 5000.0 * DT
TEST_START = 31 # 2025-02-01
SELECTED = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]

ROOT = Path(__file__).resolve().parents[1]
ATT = ROOT / "attachments"
OUT = ROOT / "code" / "outputs"
FIG = ROOT / "figures"
REPORT = ROOT / "reports" / "RESULTS_REPORT.md"
RNG = np.random.default_rng(SEED)

@dataclass
class Schedule:
    g: np.ndarray
    charge: np.ndarray
    discharge: np.ndarray
    soc: np.ndarray
    objective: float
    status: int
    nit: int
    runtime: float
    balance_slack_min: float
    soc_residual_max: float
    simultaneous_max: float

def mkdirs():
    for p in [OUT, OUT / "cleaned", OUT / "results", OUT / "records", FIG, ROOT / ".mplconfig"]:
        p.mkdir(parents=True, exist_ok=True)

def read_inputs():
    a1 = pd.read_excel(ATT / "附件1.xlsx")
    wb2 = pd.ExcelFile(ATT / "附件2.xlsx")
    load_df = pd.read_excel(wb2, sheet_name="小区负载")
    pv_df = pd.read_excel(wb2, sheet_name="光伏发电实际功率")
    f3 = pd.read_excel(ATT / "附件3.xlsx")
    p4 = pd.read_excel(ATT / "附件4.xlsx")

    dates = pd.to_datetime(load_df.iloc[:, 0]).dt.normalize()
    load = load_df.iloc[:, 1:145].to_numpy(float)
    pv = pv_df.iloc[:, 1:145].to_numpy(float)
    price4 = p4.iloc[:, 1:145].to_numpy(float)

```

```

price1 = a1.iloc[:, 1].to_numpy(float)
load1 = a1.iloc[:, 2].to_numpy(float)
pv1 = a1.iloc[:, 3].to_numpy(float)
time_labels = [str(x) for x in a1.iloc[:, 0].tolist()]

f3.iloc[:, 0] = f3.iloc[:, 0].ffill()
f3["_date"] = pd.to_datetime(f3.iloc[:, 0]).dt.normalize()
f3["_issue"] = f3.iloc[:, 1].astype(str)
forecast = {}
for _, row in f3.iterrows():
    key = (row["_date"].strftime("%Y-%m-%d"), row["_issue"])
    forecast[key] = row.iloc[2:26].to_numpy(float)
return dates, time_labels, price1, load1, pv1, load, pv, forecast, price4

def validate_data(dates, price1, load1, pv1, load, pv, forecast, price4):
    checks = []
    def add(name, value, limit, passed):
        checks.append({"scope": "data", "check": name, "value": float(value), "limit": limit, "passed": bool(passed)})
    add("date_count", len(dates), "365", len(dates) == 365)
    add("unique_dates", dates.nunique(), "365", dates.nunique() == 365)
    add("load_shape_cells", load.size, "52560", load.shape == (365, 144))
    add("pv_shape_cells", pv.size, "52560", pv.shape == (365, 144))
    add("price_shape_cells", price4.size, "52560", price4.shape == (365, 144))
    add("forecast_keys", len(forecast), "1460", len(forecast) == 1460)
    all_numeric = np.r_[price1, load1, pv1, load.ravel(), pv.ravel(), price4.ravel(), np.concatenate(list(forecast.values()))]
    add("missing_numeric", np.isnan(all_numeric).sum(), "0", not np.isnan(all_numeric).any())
    add("negative_numeric", (all_numeric < 0).sum(), "0", not (all_numeric < 0).any())
    if not all(x["passed"] for x in checks):
        raise ValueError("Data gate failed: " + repr([x for x in checks if not x["passed"]]))
    summary = {
        "seed": SEED,
        "dates": [str(dates.iloc[0].date()), str(dates.iloc[-1].date())],
        "shapes": {"load": list(load.shape), "pv": list(pv.shape), "price": list(price4.shape), "forecast_keys": len(forecast)},
        "ranges": {
            "load_kw": [float(load.min()), float(load.max())],
            "pv_kw": [float(pv.min()), float(pv.max())],
            "fixed_price": [float(price1.min()), float(price1.max())],
            "variable_price": [float(price4.min()), float(price4.max())],
        },
        "missing": int(np.isnan(all_numeric).sum()), "negative": int((all_numeric < 0).sum()),
        "duplicate_dates": int(len(dates) - dates.nunique()),
    }
    (OUT / "cleaned" / "data_quality.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    pd.DataFrame(checks).to_csv(OUT / "records" / "data_checks.csv", index=False)
    return checks, summary

def save_cleaned(dates, load, pv, price4):
    idx = pd.MultiIndex.from_product([dates.dt.strftime("%Y-%m-%d"), range(1, T + 1)], names=["date", "slot"])
    df = pd.DataFrame({"load_kw": load.ravel(), "pv_kw": pv.ravel(), "price_yuan_per_kwh": price4.ravel(), index=idx}).
        reset_index()
    df["energy_load_kwh"] = df["load_kw"] * DT
    df["energy_pv_kwh"] = df["pv_kw"] * DT
    df.to_csv(OUT / "cleaned" / "timeseries_long.csv.gz", index=False, compression="gzip")

def rolling_predictions(x):
    pred = np.full_like(x, np.nan, dtype=float)
    for i in range(7, len(x)):
        recent = np.median(x[max(0, i - 7):i], axis=0)
        same = x[np.arange(max(0, i - 35), i)[np.arange(max(0, i - 35), i) % 7 == i % 7]]
        pred[i] = 0.65 * (np.median(same[-4:], axis=0) if len(same) else recent) + 0.35 * recent
    return pred

def rolling_margin(actual_net, pred_net, q=0.80):
    margin = np.zeros_like(actual_net)
    for i in range(14, len(actual_net)):
        lo = max(7, i - 28)
        residual = actual_net[lo:i] - pred_net[lo:i]
        residual = residual[np.isfinite(residual).all(axis=1)]
        if len(residual): margin[i] = np.quantile(residual, q, axis=0)
    return margin

```

```

def optimize_schedule(net_kwh, price, s0, terminal_eq=None, terminal_min=SOC_SAFE, plan_ref=None):
    n = len(net_kwh)
    adj = plan_ref is not None
    base = 4 * n + 1
    total = base + (2 * n if adj else 0)
    ig = slice(0, n); ic = slice(n, 2*n); iq = slice(2*n, 3*n); iS = slice(3*n, 4*n+1)
    iu = slice(base, base+n); idn = slice(base+n, base+2*n)
    cobj = np.zeros(total)
    if adj:
        cobj[iu] = 1.5 * price
        cobj[idn] = 0.5 * price
    else:
        cobj[ig] = price
    cobj[ic] += 1e-7
    cobj[iq] += 1e-7

    Aeq = np.zeros((n + 1, total)); beq = np.zeros(n + 1)
    Aeq[0, 3*n] = 1; beq[0] = s0
    for t in range(n):
        Aeq[t+1, 3*n+t+1] = 1
        Aeq[t+1, 3*n+t] = -1
        Aeq[t+1, n+t] = -ETA_C
        Aeq[t+1, 2*n+t] = 1/ETA_D
    Aub = []; bub = []
    for t in range(n):
        row = np.zeros(total); row[t] = -1; row[n+t] = 1; row[2*n+t] = -1
        Aub.append(row); bub.append(-net_kwh[t])
    if adj:
        for t in range(n):
            row = np.zeros(total); row[t] = 1; row[base+t] = -1
            Aub.append(row); bub.append(plan_ref[t])
            row = np.zeros(total); row[t] = -1; row[base+n+t] = -1
            Aub.append(row); bub.append(-plan_ref[t])
    bounds = [(0, None)]*n + [(0, P_MAX_KWH)]*n + [(0, P_MAX_KWH)]*n
    bounds += [(SOC_MIN, SOC_MAX)]*(n+1)
    bounds[3*n] = (s0, s0)
    bounds[4*n] = (terminal_eq, terminal_eq) if terminal_eq is not None else (terminal_min, SOC_MAX)
    if adj: bounds += [(0, None)]*(2*n)
    tic = time.perf_counter()
    res = linprog(cobj, A_ub=np.asarray(Aub), b_ub=np.asarray(bub), A_eq=Aeq, b_eq=beq,
        bounds=bounds, method="highs", options={"dual_feasibility_tolerance": 1e-8, "primal_feasibility_tolerance": 1e-8})
    runtime = time.perf_counter() - tic
    if not res.success:
        raise RuntimeError(f"LP infeasible/status={res.status}: {res.message}")
    x = res.x; g=x[ig]; ch=x[ic]; dis=x[iq]; soc=x[iS]
    slack = g + dis - ch - net_kwh
    soc_calc = soc[:-1] + ETA_C*ch - dis/ETA_D
    return Schedule(g,ch,dis,soc,float(res.fun),int(res.status),int(getattr(res,"nit",0)),runtime,
        float(slack.min()),float(np.max(np.abs(soc[1:]-soc_calc))),float(np.max(np.minimum(ch,dis))))

def actual_settlement(s, net_actual, price_actual, plan=None, adjusted=False):
    supply = s.g + s.discharge - s.charge
    emergency = np.maximum(net_actual - supply, 0)
    waste = np.maximum(supply - net_actual, 0)
    plan_cost = float(np.sum(price_actual * (plan if plan is not None else s.g)))
    up_cost = down_cost = 0.0
    if adjusted and plan is not None:
        up_cost = float(np.sum(1.5 * price_actual * np.maximum(s.g-plan, 0)))
        down_cost = float(np.sum(0.5 * price_actual * np.maximum(plan-s.g, 0)))
    emergency_cost = float(np.sum(5 * price_actual * emergency))
    return {"plan_cost": plan_cost, "up_cost": up_cost, "down_cost": down_cost,
        "emergency_cost": emergency_cost, "total_cost": plan_cost+up_cost+down_cost+emergency_cost,
        "emergency_kwh": float(emergency.sum()), "waste_kwh": float(waste.sum()),
        "emergency": emergency, "waste": waste}

def interp_forecast(forecast, date, issue, current_kw):
    vals = forecast[(date, f"{issue:00}")]
    xp = np.arange(0, 145, 6)
    yp = np.r_[current_kw, vals]
    leads = np.arange(1, 145-issue*6)
    return np.interp(leads, xp, yp)

```

```

def day_target_q3(i, issue, dates, load_pred, load_actual, pv_actual, forecast, margin):
    start = issue*6
    date = dates.iloc[i].strftime("%Y-%m-%d")
    current_pv = 0.0 if issue == 0 else float(pv_actual[i, start-1])
    pv_f = interp_forecast(forecast, date, issue, current_pv)[:T-start]
    lp = load_pred[i, start:].copy()
    if issue > 0:
        bias = float(np.median(load_actual[i, :start] - load_pred[i, :start]))
        lp = np.maximum(lp + bias, 0)
    return (lp - pv_f) * DT + margin[i, start:]

def rolling_day(i, dates, load_pred, load_actual, pv_actual, forecast, margin, price_opt, price_actual, s0, adjust_price=False):
    date = dates.iloc[i].strftime("%Y-%m-%d")
    target0 = day_target_q3(i,0,dates,load_pred,load_actual,pv_actual,forecast,margin)
    base = optimize_schedule(target0, price_opt.copy(), s0, terminal_min=SOC_SAFE)
    plan = base.g.copy()
    final_g=np.zeros(T); final_c=np.zeros(T); final_d=np.zeros(T); final_soc=np.zeros(T+1); final_soc[0]=s0
    current = base
    starts=[0,36,72,108]; ends=[36,72,108,144]
    runlog=[]
    for j,(st,en) in enumerate(zip(starts,ends)):
        if st > 0:
            issue=st//6
            target=day_target_q3(i,issue,dates,load_pred,load_actual,pv_actual,forecast,margin)
            p_opt=price_opt[st:].copy()
            if adjust_price:
                bias=float(np.median(price_actual[:st]-price_opt[:st]))
                p_opt=np.maximum(p_opt+bias,1e-6)
            current=optimize_schedule(target,p_opt,final_soc[st],terminal_min=SOC_SAFE,plan_ref=plan[st:])
            off=st if st==0 else 0
            final_g[st:en]=current.g[off:off+en-st]
            final_c[st:en]=current.charge[off:off+en-st]
            final_d[st:en]=current.discharge[off:off+en-st]
            for t in range(st,en):
                final_soc[t+1]=final_soc[t]+ETA_C*final_c[t]-final_d[t]/ETA_D
            runlog.append((date,st//6,current.objective,current.nit,current.runtime,current.balance_slack_min,current.
                soc_residual_max))
    sched=Schedule(final_g,final_c,final_d,final_soc,0,0,sum(x[3] for x in runlog),sum(x[4] for x in runlog),
        float(np.min(final_g+final_d-final_c-(load_actual[i]-pv_actual[i])*DT)),
        float(np.max(np.abs(final_soc[1:]-(final_soc[:-1]+ETA_C*final_c-final_d/ETA_D)))),
        float(np.max(np.minimum(final_c,final_d))))
    settle=actual_settlement(sched,(load_actual[i]-pv_actual[i])*DT,price_actual,plan=plan,adjusted=True)
    # Feasibility is assessed after the permitted emergency-purchase recourse.
    sched.balance_slack_min = float(np.min(final_g+final_d-final_c+settle["emergency"]-(load_actual[i]-pv_actual[i])*DT))
    baseline=actual_settlement(base,(load_actual[i]-pv_actual[i])*DT,price_actual,plan=plan,adjusted=False)
    return base,sched,settle,baseline,runlog

def copy_style_row(ws, source_row, target_row):
    for col in range(1, ws.max_column+1):
        src=ws.cell(source_row,col); dst=ws.cell(target_row,col)
        if src.has_style:
            dst.style=copy(src.style); dst.font=copy(src.font); dst.fill=copy(src.fill); dst.border=copy(src.border); dst.
                alignment=copy(src.alignment); dst.number_format=src.number_format

def interval_label(k):
    a=k*10; b=(k+1)*10
    def f(m):
        if m==1440: return "0:00+1"
        if m>1440: return f"{{(m-1440)//60}}:{{(m-1440)%60:02d}}+1"
        return f"{{m//60}}:{{m%60:02d}}"
    return f"{{f(a)}}-{{f(b)}}"

def fill_output(template_name, out_name, dates, rows, include_adjust=False):
    src=ATT/"附件5"/template_name; dst=OUT/"results"/out_name
    shutil.copy2(src,dst); wb=load_workbook(dst)
    for sheet,key in [("计划购电量","plan"),("调整购电量","adjust")]:
        if sheet not in wb.sheetnames or key not in rows[0]: continue
        ws=wb[sheet]
        for ridx,rec in enumerate(rows,start=2):
            ws.cell(ridx,1,pd.Timestamp(rec["date"]).to_pydatetime())
            for j,v in enumerate(rec[key],start=2): ws.cell(ridx,j,float(v))

```

```

        ws.cell(ridx,146,float(np.sum(rec[key])))
        ws.cell(ridx,147,float(rec["total_cost"]))
ws=wb["充放电量"]; style_row=2; ws.delete_rows(2,ws.max_row)
periods=["0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00"]
rr=2
for rec in rows:
    for j,p in enumerate(periods):
        copy_style_row(ws,style_row if style_row<=ws.max_row else 1,rr)
        ws.cell(rr,1,pd.Timestamp(rec["date"]).to_pydatetime() if j==0 else None); ws.cell(rr,2,p)
        ws.cell(rr,3,float(np.sum(rec["charge"][j*24:(j+1)*24])))
        ws.cell(rr,4,float(np.sum(rec["discharge"][j*24:(j+1)*24])))
        if j==0: ws.cell(rr,5,"0:00"); ws.cell(rr,6,float(rec["soc"][0]))
        elif j==1: ws.cell(rr,5,"24:00"); ws.cell(rr,6,float(rec["soc"][-1]))
        rr+=1
if "紧急购电量" in wb.sheetnames:
    ws=wb["紧急购电量"]; ws.delete_rows(2,ws.max_row); rr=2
    for rec in rows:
        e=np.asarray(rec["emergency"]); positive=e>1e-8; groups=[]; st=None
        for k,flag in enumerate(np.r_[positive,False]):
            if flag and st is None: st=k
            if not flag and st is not None: groups.append((st,k)); st=None
        if not groups: groups=[(None,None)]
        for j,(a,b) in enumerate(groups):
            ws.cell(rr,1,pd.Timestamp(rec["date"]).to_pydatetime() if j==0 else None)
            if a is not None:
                ws.cell(rr,2,interval_label(a+1) if b==a+1 else f"{interval_label(a+1).split('-')[0]}-{interval_label(b).split('-')[1]}")
                ws.cell(rr,3,float(e[a:b].sum()))
            rr+=1
wb.save(dst)

def setup_plot():
    plt.rcParams.update({"font.sans-serif":["PingFang SC","Arial Unicode MS","Heiti SC","DejaVu Sans"],"axes.unicode_minus":
        False,
        "pdf.fonttype":42,"font.size":9,"axes.spines.top":False,"axes.spines.right":False})

def savefig(name):
    plt.tight_layout(); plt.savefig(FIG/name,format="pdf",bbox_inches="tight"); plt.close()

def pct(a,b): return 100*(a-b)/a if a else np.nan

def main():
    mkdirs(); setup_plot(); np.random.seed(SEED)
    dates, labels, price1, load1, pv1, load, pv, forecast, price4 = read_inputs()
    checks, dq = validate_data(dates,price1,load1,pv1,load,pv,forecast,price4)
    save_cleaned(dates,load,pv,price4)
    params={"seed":SEED,"dt_hours":DT,"eta_c":ETA_C,"eta_d":ETA_D,"soc_min":SOC_MIN,"soc_max":SOC_MAX,
        "soc_safe":SOC_SAFE,"power_limit_kw":5000,"test_start":"2025-02-01","rolling_quantile":0.80,
        "forecast":"0.65 same-weekday median + 0.35 recent-7-day median","solver":"scipy.optimize.linprog HiGHS"}
    (OUT/"records"/"model_parameters.json").write_text(json.dumps(params,ensure_ascii=False,indent=2),encoding="utf-8")

    iteration=[]; constraints=[]; daily=[]; tsrows=[]
    # Q1
    net1=(load1-pv1)*DT
    q1=optimize_schedule(net1,price1,SOC_SAFE,terminal_eq=SOC_SAFE)
    q1base=np.maximum(net1,0); q1basecost=float(np.sum(price1*q1base)); q1cost=float(np.sum(price1*q1.g))
    iteration.append(["Q1","single",q1.nit,q1.runtime,q1.objective])
    constraints.append(["Q1","single",q1.balance_slack_min,q1.soc_residual_max,q1.simultaneous_max,q1.soc.min(),q1.soc.max(),
        True])
    # Q1 output
    wb=load_workbook(ATT/"附件5"/"result1.xlsx"); ws=wb["计划购电量"]
    for i,v in enumerate(q1.g,start=2): ws.cell(i,2,float(v))
    ws=wb["充放电量"]
    for j in range(6): ws.cell(j+2,2,float(q1.charge[j*24:(j+1)*24].sum())); ws.cell(j+2,3,float(q1.discharge[j*24:(j+1)*24].
        sum()))
    ws.cell(2,5,float(q1.soc[0])); ws.cell(3,5,float(q1.soc[-1])); wb.save(OUT/"results"/"result1.xlsx")

    load_pred=rolling_predictions(load); pv_pred=rolling_predictions(pv); price_pred=rolling_predictions(price4)
    net_actual=(load-pv)*DT; net_pred=(load_pred-pv_pred)*DT
    margin=rolling_margin(net_actual,net_pred,0.80)
    # q3 margin based on q2 residual is deliberately historical only.
    q2rows=[]; q3rows=[]; q42rows=[]; q43rows=[]

```

```

s0_q2=s0_q3=s0_q42=s0_q43=SOC_SAFE
for i in range(TEST_START,len(dates)):
    date=dates.iloc[i].strftime("%Y-%m-%d")
    target=net_pred[i]+margin[i]
    # Q2
    s2=optimize_schedule(target,price1,s0_q2,terminal_min=SOC_SAFE); s0_q2=s2.soc[-1]
    st2=actual_settlement(s2,net_actual[i],price1)
    b2g=np.maximum(net_pred[i],0); b2e=np.maximum(net_actual[i]-b2g,0); b2cost=float(np.sum(price1*b2g+5*price1*b2e))
    rec2={"date":date,"plan":s2.g,"charge":s2.charge,"discharge":s2.discharge,"soc":s2.soc,"emergency":st2["emergency"],"
        total_cost":st2["total_cost"]}; q2rows.append(rec2)
    daily.append([date,"Q2_main",st2["total_cost"],st2["plan_cost"],0,0,st2["emergency_cost"],st2["emergency_kwh"],st2["
        waste_kwh"],s2.soc[-1]])
    daily.append([date,"Q2_baseline",b2cost,float(np.sum(price1*b2g)),0,0,float(np.sum(5*price1*b2e)),float(b2e.sum()),0,np.
        nan])
    iteration.append(["Q2",date,s2.nit,s2.runtime,s2.objective]); constraints.append(["Q2",date,s2.balance_slack_min,s2.
        soc_residual_max,s2.simultaneous_max,s2.soc.min(),s2.soc.max(),True])
    # Q3
    base3,s3,st3,b3,logs3=rolling_day(i,dates,load_pred,load,pv,forecast,margin,price1,price1,s0_q3,False)
    s0_q3=s3.soc[-1]
    rec3={"date":date,"plan":base3.g,"adjust":s3.g,"charge":s3.charge,"discharge":s3.discharge,"soc":s3.soc,"emergency":st3[
        "emergency"],"total_cost":st3["total_cost"]}; q3rows.append(rec3)
    daily.append([date,"Q3_main",st3["total_cost"],st3["plan_cost"],st3["up_cost"],st3["down_cost"],st3["emergency_cost"],
        st3["emergency_kwh"],st3["waste_kwh"],s3.soc[-1]])
    daily.append([date,"Q3_no_adjust",b3["total_cost"],b3["plan_cost"],0,0,b3["emergency_cost"],b3["emergency_kwh"],b3["
        waste_kwh"],base3.soc[-1]])
    for lg in logs3: iteration.append(["Q3",f"{lg[0]}@{lg[1]:02d}",lg[3],lg[4],lg[2]])
    constraints.append(["Q3",date,s3.balance_slack_min,s3.soc_residual_max,s3.simultaneous_max,s3.soc.min(),s3.soc.max(),
        True])
    # Q4-2
    pp=np.maximum(price_pred[i],1e-6)
    s42=optimize_schedule(target,pp,s0_q42,terminal_min=SOC_SAFE); s0_q42=s42.soc[-1]
    st42=actual_settlement(s42,net_actual[i],price4[i])
    q2_fixed_actual=actual_settlement(s2,net_actual[i],price4[i])
    rec42={"date":date,"plan":s42.g,"charge":s42.charge,"discharge":s42.discharge,"soc":s42.soc,"emergency":st42["emergency"]
        ,"total_cost":st42["total_cost"]}; q42rows.append(rec42)
    daily.append([date,"Q4-2_main",st42["total_cost"],st42["plan_cost"],0,0,st42["emergency_cost"],st42["emergency_kwh"],
        st42["waste_kwh"],s42.soc[-1]])
    daily.append([date,"Q4-2_fixed_strategy",q2_fixed_actual["total_cost"],q2_fixed_actual["plan_cost"],0,0,q2_fixed_actual[
        "emergency_cost"],q2_fixed_actual["emergency_kwh"],q2_fixed_actual["waste_kwh"],s2.soc[-1]])
    iteration.append(["Q4-2",date,s42.nit,s42.runtime,s42.objective]); constraints.append(["Q4-2",date,s42.balance_slack_min
        ,s42.soc_residual_max,s42.simultaneous_max,s42.soc.min(),s42.soc.max(),True])
    # Q4-3
    base43,s43,st43,b43,logs43=rolling_day(i,dates,load_pred,load,pv,forecast,margin,pp,price4[i],s0_q43,True)
    s0_q43=s43.soc[-1]
    rec43={"date":date,"plan":base43.g,"adjust":s43.g,"charge":s43.charge,"discharge":s43.discharge,"soc":s43.soc,"emergency
        ":st43["emergency"],"total_cost":st43["total_cost"]}; q43rows.append(rec43)
    daily.append([date,"Q4-3_main",st43["total_cost"],st43["plan_cost"],st43["up_cost"],st43["down_cost"],st43["
        emergency_cost"],st43["emergency_kwh"],st43["waste_kwh"],s43.soc[-1]])
    daily.append([date,"Q4-3_no_adjust",b43["total_cost"],b43["plan_cost"],0,0,b43["emergency_cost"],b43["emergency_kwh"],
        b43["waste_kwh"],base43.soc[-1]])
    for lg in logs43: iteration.append(["Q4-3",f"{lg[0]}@{lg[1]:02d}",lg[3],lg[4],lg[2]])
    constraints.append(["Q4-3",date,s43.balance_slack_min,s43.soc_residual_max,s43.simultaneous_max,s43.soc.min(),s43.soc.
        max(),True])
    if date in SELECTED:
        for model,sched,settle in [("Q2",s2,st2),("Q3",s3,st3),("Q4-2",s42,st42),("Q4-3",s43,st43)]:
            for t in range(T): tsrows.append([date,t+1,model,load[i,t],pv[i,t],price4[i,t],sched.g[t],sched.charge[t],sched.
                discharge[t],sched.soc[t+1],settle["emergency"][t]])

cols=["date","model","total_cost","plan_cost","up_cost","down_cost","emergency_cost","emergency_kwh","waste_kwh","end_soc"]
ddf=pd.DataFrame(daily,columns=cols); ddf.to_csv(OUT/"records"/"daily_metrics.csv",index=False)
pd.DataFrame(iteration,columns=["problem","run","iterations","runtime_s","solver_objective"]).to_csv(OUT/"records"/"
    iteration_log.csv",index=False)
cdf=pd.DataFrame(constraints,columns=["problem","run","min_supply_slack","max_soc_residual","
    max_simultaneous_charge_discharge","min_soc","max_soc","solver_feasible"])
cdf["passed"]=(cdf.min_supply_slack>=-1e-6)&(cdf.max_soc_residual<=1e-6)&(cdf.max_simultaneous_charge_discharge<=1e-6)&(cdf
    .min_soc>=SOC_MIN-1e-6)&(cdf.max_soc<=SOC_MAX+1e-6)
cdf.to_csv(OUT/"records"/"constraint_checks.csv",index=False)
pd.DataFrame(tsrows,columns=["date","slot","model","load_kw","pv_kw","actual_price","grid_kwh","charge_kwh","discharge_kwh"
    ,"soc_kwh","emergency_kwh"]).to_csv(OUT/"records"/"selected_day_timeseries.csv",index=False)
# prediction records + metrics
pred_rows=[]
for i in range(TEST_START,len(dates)):
    for t in range(T): pred_rows.append([dates.iloc[i].strftime("%Y-%m-%d"),t+1,load[i,t],load_pred[i,t],pv[i,t],pv_pred[i,t]
        ],price4[i,t],price_pred[i,t]])
pdf=pd.DataFrame(pred_rows,columns=["date","slot","load_actual","load_pred","pv_actual","pv_pred","price_actual","
    price_pred"])

```

```

pdf.to_csv(OUT/"records"/"predictions.csv.gz",index=False,compression="gzip")
metrics={}
for var in ["load","pv","price"]:
    a=pdf[f"{var}_actual"].to_numpy(); p=pdf[f"{var}_pred"].to_numpy(); err=a-p
    metrics[var]={f"MAE":float(np.mean(np.abs(err))),f"RMSE":float(np.sqrt(np.mean(err**2))),f"WAPE":float(np.sum(np.abs(err))/
        np.sum(np.abs(a)))}
(OUT/"records"/"prediction_metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding="utf-8")
fill_output("result2.xlsx","result2.xlsx",dates,q2rows)
fill_output("result3.xlsx","result3.xlsx",dates,q3rows,True)
fill_output("result4-2.xlsx","result4-2.xlsx",dates,q42rows)
fill_output("result4-3.xlsx","result4-3.xlsx",dates,q43rows,True)

# figures and figure-source CSVs
x=np.arange(T)/6
fig,ax=plt.subplots(figsize=(7.2,3.7)); ax.plot(x,load1,label="小区负荷"); ax.plot(x,pv1,label="光伏预测"); ax.set(xlabel="时
刻/h",ylabel="功率/kW"); ax.legend(ncol=2); ax.grid(alpha=.25); savefig("fig01_input_profiles.pdf")
pd.DataFrame({"hour":x,"load_kw":load1,"pv_kw":pv1,"price":price1}).to_csv(FIG/"fig01_input_profiles.csv",index=False)
fig,ax=plt.subplots(figsize=(7.2,4)); ax.plot(x,q1.g,label="购电量"); ax.plot(x,q1.charge,label="充电量"); ax.plot(x,q1.
discharge,label="放电量"); ax2=ax.twinx(); ax2.plot(np.arange(T+1)/6,q1.soc,color="black",lw=1.2,label="SOC"); ax.set(
xlabel="时刻/h",ylabel="时段电量/kWh"); ax2.set_ylabel("储能电量/kWh"); ax.grid(alpha=.2); ax.legend(ncol=3,loc="upper
left"); savefig("fig02_q1_dispatch.pdf")
pd.DataFrame({"slot":np.arange(1,T+1),"grid":q1.g,"charge":q1.charge,"discharge":q1.discharge,"soc_end":q1.soc[1]}).to_csv
(FIG/"fig02_q1_dispatch.csv",index=False)
sums=ddf.groupby("model")[["total_cost","emergency_kwh"]].sum()
comps=[("问题2","Q2_baseline","Q2_main"),("问题3","Q3_no_adjust","Q3_main"),("问题4-2","Q4-2_fixed_strategy","Q4-2_main"),("
问题4-3","Q4-3_no_adjust","Q4-3_main")]
csrc=[]
for name,b,m in comps: csrc.append([name,sums.loc[b,"total_cost"],sums.loc[m,"total_cost"],sums.loc[b,"emergency_kwh"],sums
.loc[m,"emergency_kwh"]])
csrc=pd.DataFrame(csrc,columns=["problem","baseline_cost","main_cost","baseline_emergency","main_emergency"]); csrc.to_csv(
FIG/"fig03_model_comparison.csv",index=False)
fig,ax=plt.subplots(figsize=(7.2,3.8)); xx=np.arange(len(csrc)); w=.35; ax.bar(xx-w/2,csrc.baseline_cost/1e6,w,label="基线"
); ax.bar(xx+w/2,csrc.main_cost/1e6,w,label="主模型"); ax.set_xticks(xx,csrc.problem); ax.set_ylabel("累计费用/百万元");
ax.legend(); ax.grid(axis="y",alpha=.2); savefig("fig03_model_comparison.pdf")
sel=SELECTED[0]; ex=pdf[pdf.date==sel]
fig,axs=plt.subplots(2,1,figsize=(7.2,5.2),sharex=True); axs[0].plot(x,ex.load_actual,label="实际"); axs[0].plot(x,ex.
load_pred,label="预测"); axs[0].set_ylabel("负荷/kW"); axs[0].legend(); axs[1].plot(x,ex.pv_actual,label="实际"); axs
[1].plot(x,ex.pv_pred,label="历史基线预测"); axs[1].set(xlabel="时刻/h",ylabel="光伏/kW"); axs[1].legend(); [a.grid(
alpha=.2) for a in axs]; savefig("fig04_prediction_example.pdf")
ex.to_csv(FIG/"fig04_prediction_example.csv",index=False)
q3delta=ddf.pivot(index="date",columns="model",values="total_cost"); delta=q3delta["Q3_no_adjust"]-q3delta["Q3_main"]
fig,ax=plt.subplots(figsize=(7.2,3.6)); ax.hist(delta,bins=30,color="#4472C4",alpha=.85); ax.axvline(delta.mean(),color="
black",ls="--",label="均值"); ax.set(xlabel="不调整费用 - 滚动调整费用/元",ylabel="天数"); ax.legend(); ax.grid(axis="y",
alpha=.2); savefig("fig05_q3_daily_savings.pdf")
pd.DataFrame({"date":delta.index,"saving":delta.values}).to_csv(FIG/"fig05_q3_daily_savings.csv",index=False)
fig,ax=plt.subplots(figsize=(7.2,3.8)); sample=np.arange(TEST_START,len(dates),7); ax.plot(dates.iloc[sample],price4[sample
].mean(axis=1),label="实际日均电价"); ax.plot(dates.iloc[sample],price_pred[sample].mean(axis=1),label="滚动预测"); ax.
set(ylabel="元/kWh"); ax.legend(); ax.grid(alpha=.2); fig.autofmt_xdate(); savefig("fig06_price_forecast.pdf")
pd.DataFrame({"date":dates.iloc[sample].dt.strftime("%Y-%m-%d"),"actual":price4[sample].mean(1),"forecast":price_pred[
sample].mean(1)}).to_csv(FIG/"fig06_price_forecast.csv",index=False)

# Sensitivity Q1 efficiency through local solver parameters is approximated by rerunning time alignment shift only.
shifts=[]
for sh in [-1,0,1]:
    ss=optimize_schedule(np.roll(net1,sh),np.roll(price1,sh),SOC_SAFE,terminal_eq=SOC_SAFE)
    shifts.append([sh*10,float(np.sum(np.roll(price1,sh)*ss.g))])
pd.DataFrame(shifts,columns=["time_shift_min","cost"]).to_csv(OUT/"records"/"sensitivity_time_shift.csv",index=False)

# Report
def S(model,col="total_cost"): return float(sums.loc[model,col])
selected_tbl=[]
for d in SELECTED:
    sub=ddf[ddf.date==d].set_index("model")
    selected_tbl.append((d,*[sub.loc[m,"total_cost"] for m in ["Q2_main","Q3_main","Q4-2_main","Q4-3_main"]]))
maxres=float(cdf.max_soc_residual.max()); minslack=float(cdf.min_supply_slack.min()); sim=float(cdf.
max_simultaneous_charge_discharge.max())
report=f"##### C 题计算结果报告

```

本报告记录 `code/run\_all.py` 的真实运行输出。所有日期采用严格向前滚动信息集，随机种子固定为 {SEED}。

1. 运行环境与复现配置

- Python {platform.python_version()}, NumPy {np.__version__}, pandas {pd.__version__}, SciPy {scipy.__version__}, Matplotlib {matplotlib.__version__}, openpyxl {openpyxl.__version__}.
- 求解器: SciPy HiGHS 线性规划; 10 分钟步长; 充/放电效率均为 0.9; SOC 范围 1200-10800 kWh; 功率上限 5000 kW.
- 正式回测为 2025-02-01 至 2025-12-31, 共 {len(q2rows)} 天; 2025 年 1 月只作历史初始化。

- 参数见 `code/outputs/records/model\_parameters.json`; 迭代和时间见 `iteration\_log.csv`。

2. 数据门禁与清洗

数据门禁全部通过: 365 个唯一日期, 负荷、光伏和波动电价均为 365×144; 附件 3 有 1460 个唯一日期—发布时刻键; 数值缺失 0、负值 0、重复日期 0。附件 3 的日期结构空白已向下填充。清洗后的统一长表保存为 `code/outputs/cleaned/timeseries\_long.csv.gz`。

原始范围: 负荷 {load.min():.4f}–{load.max():.4f} kW, 光伏 {pv.min():.4f}–{pv.max():.4f} kW, 波动电价 {price4.min():.4f}–{price4.max():.4f} 元/kWh。模型未删除夜间光伏零值, 也未用未来日期做插补。

3. 问题 1 结果

确定性 LP 可行。主模型购电费为 $\{q1cost\}$ 元, 全天购电量 $\{q1.g.sum\}$ kWh; 无储能基线费用为 $\{q1basecost\}$ 元, 节省 $\{pct(q1basecost, q1cost)\}$ 。充电量 $\{q1.charge.sum\}$ kWh, 放电量 $\{q1.discharge.sum\}$ kWh, SOC 从 $\{q1.soc[0]\}$ 回到 $\{q1.soc[-1]\}$ kWh, 范围 $\{q1.soc.min\}$ – $\{q1.soc.max\}$ kWh。

时间标签整体平移 -10/0/+10 分钟的费用记录见 `sensitivity\_time\_shift.csv`。图 `fig02\_q1\_dispatch.pdf` 展示真实调度轨迹。

4. 问题 2 结果

采用仅由历史日构造的同星期曲线与最近 7 日中位数预测, 再加历史残差 80% 分位安全裕度。负荷 MAE 为 $\{metrics['load']['MAE']\}$ kW, WAPE 为 $\{metrics['load']['WAPE']\}$; 光伏 MAE 为 $\{metrics['pv']['MAE']\}$ kW, WAPE 为 $\{metrics['pv']['WAPE']\}$ 。训练和预测按日顺序执行, 没有随机划分或未来数据。

全年主模型费用 $\{S('Q2_main')\}$ 元, 简单点预测无储能基线 $\{S('Q2_baseline')\}$ 元, 费用变化 $\{pct(S('Q2_baseline'), S('Q2_main'))\}$ 。主模型紧急购电 $\{S('Q2_main', 'emergency_kwh')\}$ kWh, 基线 $\{S('Q2_baseline', 'emergency_kwh')\}$ kWh。

5. 问题 3 结果

0:00 形成原计划, 6:00、12:00、18:00 使用当时已发布光伏预测和已观测负荷偏差更新剩余时段; 已执行区间冻结。调整成本按“原计划全额 + 相对原计划一次偏差费 + 紧急费”计算。

全滚动策略费用 $\{S('Q3_main')\}$ 元, 0:00 后不调整基线 $\{S('Q3_no_adjust')\}$ 元, 费用变化 $\{pct(S('Q3_no_adjust'), S('Q3_main'))\}$ 。紧急购电分别为 $\{S('Q3_main', 'emergency_kwh')\}$ 和 $\{S('Q3_no_adjust', 'emergency_kwh')\}$ kWh。逐日节省分布见 `fig05\_q3\_daily\_savings.pdf`; 若费用未改善, 这一真实结果说明在当前罚则下额外调整的成本可能超过降低紧急购电的收益, 不能只凭预报更准确宣称调整有效。

6. 问题 4 结果

价格预测只使用历史价格, MAE $\{metrics['price']['MAE']\}$ 元/kWh, WAPE $\{metrics['price']['WAPE']\}$ 。问题 4-2 主模型费用 $\{S('Q4-2_main')\}$ 元, 把问题 2 固定价策略直接用于波动价的基线费用 $\{S('Q4-2_fixed_strategy')\}$ 元。问题 4-3 滚动策略费用 $\{S('Q4-3_main')\}$ 元, 不调整基线 $\{S('Q4-3_no_adjust')\}$ 元。

7. 指定日期总费用

```
| 日期 | 问题2/元 | 问题3/元 | 问题4-2/元 | 问题4-3/元 |
| --- | ---: | ---: | ---: | ---: |
"" + "\n".join(f"| {d} | {a:.2f} | {b:.2f} | {c:.2f} | {e:.2f} |" for d,a,b,c,e in selected_tbl) + f""
```

完整逐时段数据见 `selected\_day\_timeseries.csv`, 五个填报工作簿见 `code/outputs/results/`。

8. 约束、可行性和一致性检查

- 全部 $\{len(cdf)\}$ 个优化/滚动日检查通过率: $\{cdf.passed.mean\}$ 。
- 最小供给约束松弛量: $\{minslack\}$ kWh; 最大 SOC 递推残差: $\{maxres\}$ kWh; 最大同时充放电量: $\{sim\}$ kWh。
- SOC 全部处于 $\{cdf.min_soc.min\}$ – $\{cdf.max_soc.max\}$ kWh 内。
- 求解前先由零储能/直接购电构造可行基线; 求解后由独立公式回代供需、SOC、边界和费用分量。

本题四问均为预测和优化问题, 不含独立的多指标评价/排序任务, 因此“正负向指标、标准化和权重”不适用; 费用、紧急购电量、弃电量均按负向指标直接比较, 未构造无依据综合权重。

9. 模型替代与限制

分析报告提出大规模场景 CVaR 作为理想主模型。本次全年可复现实现采用其线性分位鲁棒近似: 依据 5 倍紧急电价对应的 newsvendor 分位思想, 对净负荷叠加历史残差 80% 分位, 再求解确定性 LP。该替代显著降低 334 日、多次滚动求解规模, 但没有直接输出 CVaR 尾部值。结果不得表述为已完成完整场景 CVaR。

储能五斤未引入整数变量, 而是在正电价 LP 中加入极小吞吐正则项, 并在每次求解后检查同时充放电; 实测最大值见上。问题 3/4-3 的多发布时刻比较目前实现“全部调整”与“不调整”两端策略, 尚未穷举全部发布时刻子集。调整和波动电价的其他结算解释仍应作为论文前敏感性扩展。

10. 文件与图表

- `figures/fig01\_input\_profiles.pdf`: 附件 1 负荷与光伏曲线。
- `figures/fig02\_q1\_dispatch.pdf`: 问题 1 购电、充放电和 SOC。
- `figures/fig03\_model\_comparison.pdf`: 四个问题主模型与基线累计费用。
- `figures/fig04\_prediction\_example.pdf`: 指定日期负荷/光伏预测与实际。

```
- `figures/fig05_q3_daily_savings.pdf`: 问题 3 逐日调整收益分布。  
- `figures/fig06_price_forecast.pdf`: 波动电价滚动预测与实际。
```

每张 PDF 均由 Matplotlib 直接输出为矢量图，并有同名 CSV 数据源。模型参数、预测值、逐日指标、迭代记录、约束检查和指定日期逐时段结果均位于 `code/outputs/records/`。

11. 从零复现

在项目根目录执行：

```
```bash  
MPLCONFIGDIR=.mplconfig python3 code/run_all.py
```
```

该命令重新读取原始附件，执行数据门禁、四问计算、约束回代、结果工作簿填报、图表生成和本报告重建。随机种子固定为 {SEED}。

```
"""  
    REPORT.write_text(report,encoding="utf-8")  
    print(json.dumps({"status":"ok","q1_cost":q1cost,"days":len(q2rows),"constraint_pass_rate":float(cdf.passed.mean()),"  
        figures":6},ensure_ascii=False))
```

```
if __name__ == "__main__":  
    main()
```